HardWhale 학습 주석본

한 줄 요약

컨테이너 네트워크 보안(정책 집행)을 호스트 OS가 아닌 SmartNIC(BlueField-2) 안으로 옮겨서, 호스트 OS가 뚫려도 옆 노드로 번지는 공격(lateral movement)을 막고, 검사는 NIC의 하드웨어 가속기로 처리해 Cilium보다 HTTP 처리량 3배, 지연 2.3배, CPU 사용 4배를 개선한다. 출처: You, Nam, Seo, Seo, Kim, Choi, Shin, IEEE ICDCS 2024. 유명성 교수님 1저자.

읽는 순서

1) 초록과 1장 (문제: 호스트 OS에 의존하는 보안) 2) 2장 배경, 특히 그림 1(컨테이너 네트워크)과 그림 2(Cilium 공격 시나리오) 3) 3장 설계는 그림 3(전체 구조)과 그림 4(패킷 파이프라인)를 옆에 두고 4) 5장 보안 평가는 그림 5~8 캡션만 먼저 5) 6장 성능은 그림 9와 표 I·II 6) 8장 한계. 처음 읽을 때는 노란색만 따라가고, 두 번째에 초록 용어 메모를 읽는 것을 권한다.

표시 규칙

노랑: 핵심 주장, 결과 수치

초록: 전문용어. 오른쪽에 풀이

분홍: 한계, 가정, 위협 모델

①②… 빨간 번호 = 오른쪽 여백의 같은 번호 설명. → 표시 = EN683·한화비전과 연결되는 지점.

이 판에는 원문 쪽이 없다. 원문을 옆에 띄워 놓고 쪽 번호와 메모 번호로 따라 읽는다. 형광펜 문구는 원문에서 그대로 옮겨 적었다. 원문: https://doi.org/10.1109/ICDCS60910.2024.00053

용어집 (등장 순서)

마이크로서비스: 큰 애플리케이션을 작은 독립 모듈(컨테이너) 여러 개로 쪼개서 네트워크(주로 HTTP API)로 서로 부르게 만든 구조.

SmartNIC(=DPU): 자체 CPU(ARM 코어), 메모리, 하드웨어 가속기를 가진 네트워크 카드. 호스트 CPU 대신 네트워크 처리를 떠맡는다.

공유 커널 모델: 컨테이너는 VM과 달리 호스트 커널 하나를 같이 쓴다. 커널 취약점 하나로 모든 컨테이너가 위험해진다.

lateral movement(횡적 이동): 하나를 장악한 공격자가 네트워크를 타고 옆의 다른 컨테이너·서버로 퍼져 나가는 공격 단계.

APT(지능형 지속 위협): 오래 숨어서 내부로 깊이 침투해 데이터를 빼내는 표적 공격.

line rate: 네트워크 선로의 최대 속도(예: 40Gbps) 그대로 처리한다는 뜻. 검사 때문에 느려지지 않는다.

SID(보안 신원): 컨테이너 IP는 자주 바뀌므로 역할(frontend, backend, DB 등)에 고정 번호를 붙여 정책을 SID 기준으로 쓴다.

BlueField-2: NVIDIA(구 Mellanox)의 DPU. ARM Cortex-A72 코어 8개, 정규식·암호 가속기, 최대 100/200Gbps 포트(일반 사양, 원문 외).

network namespace(net-NS): 리눅스 커널이 컨테이너마다 네트워크 인터페이스·라우팅 표를 따로 보이게 하는 격리 기능.

bridge: 같은 노드 안의 컨테이너들을 묶는 가상 스위치. 컨테이너 쪽 인터페이스와 짝을 이룬 veth로 연결된다.

VXLAN: 컨테이너 패킷을 UDP 패킷 안에 한 번 더 싸서(캡슐화) 다른 노드로 보내는 터널 프로토콜.

eBPF: 커널 안에서 검증된 작은 프로그램을 돌리는 기능. Cilium은 이것으로 패킷을 거른다. → BEACON은 이것으로 시스템콜을 수집한다.

Envoy: 사용자 공간에서 도는 L7 프록시. HTTP 경로 같은 내용까지 보려면 패킷을 커널에서 여기로 넘겨야 해서 느리다.

L3~L7: L3=IP 주소, L4=TCP/UDP 포트, L7=HTTP 메서드·경로 같은 애플리케이션 내용. 층이 높을수록 검사가 비싸다.

pre-shared 인증서: 미리 나눠 가진 인증서로 서로를 확인한다. → 한화비전 장비의 공장 출하 인증서와 같은 발상(추정).

eBPF map: 커널의 eBPF 프로그램이 쓰는 키-값 저장소. 루트 권한이 있으면 바깥에서 읽고 쓸 수 있다.

restricted mode(BlueField의 zero-trust 모드): 호스트가 NIC 내부 설정·펌웨어·프로그램에 접근하지 못하게 막는 하드웨어 기능.

HNIC: 각 노드의 SmartNIC 위에서 도는 HardWhale 보안 스택. 하드웨어 엔진 3개(secure switch, inspection engine, policy engine)로 구성.

manager: 마스터 노드에서 도는 중앙 관리자. 컨테이너가 생기거나 바뀌면 정책을 계산해 각 HNIC로 보낸다.

secure switch(그림 3): L3/L4 정책과 신원(SID·IP) 위조 검사를 하는 HNIC의 첫 단계.

TLS: 통신을 암호화하고 상대를 인증하는 프로토콜(HTTPS의 S). 정책 업데이트 패킷을 보호한다. → 한화비전 카메라-클라우드 연결도 TLS 1.2/1.3.

secure boot: 부팅 때 펌웨어·OS·프로그램의 서명을 검증해서 변조된 소프트웨어는 실행하지 않는다. → EN683 보안 부트와 같은 개념.

agent: 노드 OS에서 도는 보조 소프트웨어. 같은 노드 안 컨테이너 사이 트래픽도 NIC를 거치게 연결한다.

CNI: 쿠버네티스가 컨테이너를 만들 때 네트워크를 붙여 주는 플러그인 규격. Cilium, Calico가 대표 CNI.

SR-IOV VF: 물리 NIC 하나를 여러 개의 가상 NIC(Virtual Function)로 나눠서 컨테이너에 직접 꽂는 기술. 커널 브리지를 건너뛴다.

스푸핑 방지: VF가 컨테이너가 보낸 패킷의 출발지 주소가 그 컨테이너 주소와 같은지 확인하고 다르면 버린다.

노드 단위 배포: 각 HNIC는 자기 노드 컨테이너에 관련된 정책만 갖는다. NIC 메모리가 작기 때문.

pre-filtering: 헤더 파싱, 연결 추적, 관련 없는 패킷 걸러내기를 하는 첫 파이프라인 단계.

decapsulation: VXLAN으로 싸서 온 패킷의 바깥 헤더를 벗겨 원래 컨테이너 패킷을 꺼내는 일.

stateful 검사: 연결(TCP 세션) 상태를 기억해서, 이미 허용된 연결의 후속 패킷은 L3/L4 검사를 건너뛴다.

match-action table: 패킷 필드가 표의 키와 맞으면(match) 정해진 동작(허용·차단)을 하는 하드웨어 스위칭 표 구조.

SID별 정책 분할: 정책 표를 SID마다 따로 둬서 조회할 표가 작아지고 빠르다.

RSS: 패킷 헤더 해시로 큐를 골라 여러 코어에 나눠 주는 기법. 같은 연결은 항상 같은 코어로 가서 캐시 효율이 좋다.

OoO 패킷: 순서가 뒤바뀌어 도착한 TCP 조각. 순서를 맞춰야 패킷 여러 개에 걸친 공격 패턴을 찾을 수 있다.

탐지 시그니처: 알려진 익스플로잇의 바이트 패턴. 페이로드 안에서 찾으면 버린다. IDS(Snort 등)의 방식과 같다.

policy engine: 매니저가 보낸 정책 업데이트를 실시간으로 반영하는 HNIC의 세 번째 엔진.

동시 업데이트: 코어마다 업데이트 큐를 두고 같은 표 항목은 항상 같은 코어가 맡게 해서 락(lock) 없이 병렬로 고친다.

ARM 코어: BlueField-2 안의 범용 CPU. 보안 모듈이 여기서 돌고, 가속기를 호출한다. → EN683도 ARM 기반 SoC라 같은 툴체인 감각이 쓰인다(추정).

TC eBPF: 리눅스 트래픽 제어(tc) 계층에 붙이는 eBPF 프로그램. 노드 내부 위조 검사를 VF마다 이것으로 구현했다.

쿠버네티스: 여러 노드에 컨테이너를 배치·관리하는 오케스트레이터. 실험은 v1.27.3, Ubuntu 22.04, Cilium.

버퍼 오버플로: 정해진 크기보다 긴 입력으로 메모리를 덮어써서 프로그램 흐름을 빼앗는 공격. 실험은 memcached 취약점 CVE-2016-8704.

Calico: iptables 기반의 대표 CNI. 역시 정책이 노드 OS에 있다.

iptables: 리눅스 커널의 전통적인 패킷 필터 규칙. 루트 권한이면 지우거나 바꿀 수 있다.

암호화+인증: 내용을 못 보게 하고(기밀성), 위조하면 드러나게 한다(무결성).

실험 환경: 노드 2대(Xeon 4114, 64GB RAM)를 40Gbps 이더넷으로 연결.

wrk2: 일정한 요청률(RPS)로 HTTP 부하를 거는 벤치마크 도구. 지연 분포 측정에 쓴다.

P99 지연: 요청 100개 중 가장 느린 1개 수준의 지연. 꼬리 지연을 본다. HardWhale 2.489ms, Cilium 12.4ms(4.9배).

다중 패턴 매칭: 여러 정규식·문자열을 한 번의 스캔으로 동시에 찾는 알고리즘(Aho-Corasick류). 정책 수가 늘어도 느려지지 않는다.

Linkerd, Istio: 서비스 메시. 컨테이너마다 사이드카 프록시를 붙여 L7 정책과 mTLS를 처리한다.

P4 스위치: 패킷 처리 파이프라인을 프로그래밍할 수 있는 스위치 칩. 데이터 평면 오프로드 연구에 자주 쓰인다.

AWS Nitro: 가상화 기능을 전용 PCIe 카드로 옮긴 AWS의 하드웨어. 강하지만 AWS 전용이라 일반 클라우드에 쓸 수 없다.

참고 [13]: BlueField DPU zero-trust 모드 = 본문의 restricted mode.

참고 [11]: HELIOS(SoCC 2023)는 같은 저자들의 선행 연구. 하드웨어 보조 클라우드 네트워크 보안 확장.

원문 1쪽

microservices
① **마이크로서비스:** 큰 애플리케이션을 작은 독립 모듈(컨테이너) 여러 개로 쪼개서 네트워크(주로 HTTP API)로 서로 부르게 만든 구조.

host's operating system (OS)
② **문제의 핵심:** 기존 보안 솔루션은 호스트 OS 안에서 정책을 집행한다. 호스트 OS가 뚫리면 보안 장치도 같이 뚫린다.

smartNIC
③ **SmartNIC(=DPU):** 자체 CPU(ARM 코어), 메모리, 하드웨어 가속기를 가진 네트워크 카드. 호스트 CPU 대신 네트워크 처리를 떠맡는다.

improves HTTP throughput threefold
④ **핵심 결과:** 최신 솔루션(Cilium) 대비 HTTP 처리량 3배, HTTP 지연 2.3배 개선.

shared kernel model
⑤ **공유 커널 모델:** 컨테이너는 VM과 달리 호스트 커널 하나를 같이 쓴다. 커널 취약점 하나로 모든 컨테이너가 위험해진다.

lateral movement
⑥ **lateral movement(횡적 이동):** 하나를 장악한 공격자가 네트워크를 타고 옆의 다른 컨테이너·서버로 퍼져 나가는 공격 단계.

advanced persistent threats
⑦ **APT(지능형 지속 위협):** 오래 숨어서 내부로 깊이 침투해 데이터를 빼내는 표적 공격.

container escape
⑧ **container escape:** 컨테이너 격리를 깨고 호스트 OS 권한을 얻는 공격. 실제로 자주 일어난다는 것이 이 논문의 전제.

line-rate performance
⑨ **line rate:** 네트워크 선로의 최대 속도(예: 40Gbps) 그대로 처리한다는 뜻. 검사 때문에 느려지지 않는다.

identity verification
⑩ 패킷마다 보낸 컨테이너의 신원을 확인한다. 위조 신원(SID spoofing)을 막는 장치.

원문 2쪽

Security identity (SID)
① **SID(보안 신원):** 컨테이너 IP는 자주 바뀌므로 역할(frontend, backend, DB 등)에 고정 번호를 붙여 정책을 SID 기준으로 쓴다.

hardware-
② **핵심 설계:** SmartNIC의 하드웨어 기능(restricted mode, secure boot)으로 호스트 OS가 NIC 설정을 건드리지 못하게 한다.

Mellanox Bluefield2
③ **BlueField-2:** NVIDIA(구 Mellanox)의 DPU. ARM Cortex-A72 코어 8개, 정규식·암호 가속기, 최대 100/200Gbps 포트(일반 사양, 원문 외).

5.7x
④ TCP 지연 최대 5.7배, HTTP 지연 최대 2.6배 개선.

network names-
⑤ **network namespace(net-NS):** 리눅스 커널이 컨테이너마다 네트워크 인터페이스·라우팅 표를 따로 보이게 하는 격리 기능.

bridge network
⑥ **bridge:** 같은 노드 안의 컨테이너들을 묶는 가상 스위치. 컨테이너 쪽 인터페이스와 짝을 이룬 veth로 연결된다.

VXLAN
⑦ **VXLAN:** 컨테이너 패킷을 UDP 패킷 안에 한 번 더 싸서(캡슐화) 다른 노드로 보내는 터널 프로토콜.

eBPF [18]
⑧ **eBPF:** 커널 안에서 검증된 작은 프로그램을 돌리는 기능. Cilium은 이것으로 패킷을 거른다. **→ BEACON은 이것으로 시스템콜을 수집한다.**

envoy [19]
⑨ **Envoy:** 사용자 공간에서 도는 L7 프록시. HTTP 경로 같은 내용까지 보려면 패킷을 커널에서 여기로 넘겨야 해서 느리다.

L3 to L7
⑩ **L3~L7:** L3=IP 주소, L4=TCP/UDP 포트, L7=HTTP 메서드·경로 같은 애플리케이션 내용. 층이 높을수록 검사가 비싸다.

container compromise (C-comp)
⑪ **C-comp:** 컨테이너 하나만 뚫린 경우. 기존 연구들이 다룬 위협 모델.

원문 3쪽

Container-A1
① **그림 2 시나리오:** A1 탈취 **→ 노드 A 장악 → 정책 오염 → 위조 SID로 노드 B의 컨테이너 공격.** 기존 방식의 실패 사례.

eBPF map

② **eBPF map:** 커널의 eBPF 프로그램이 쓰는 키-값 저장소. 루트 권한이 있으면 바깥에서 읽고 쓸 수 있다.

poisoned policy
③ **정책 오염:** 노드를 장악한 공격자가 eBPF map에 저장된 Cilium 정책을 모두 허용으로 바꾼다.

node compro-
④ **N-comp:** 노드(호스트 OS)까지 뚫린 경우. 이 논문이 새로 제시하는 더 강한 위협 모델. 노드의 모든 컨테이너 트래픽을 보고 조작할 수 있다.

pre-shared certificates
⑤ **pre-shared 인증서:** 미리 나눠 가진 인증서로 서로를 확인한다. → **한화비전 장비의 공장 출하 인증서와 같은 발상(추정).**

trustworthy and exclude
⑥ **가정:** 마스터 노드(쿠버네티스 제어부)는 믿는다. 워커 노드만 공격 대상으로 본다.

Lack of isolation for policy enforcement stack
⑦ **한계 1:** 보안 장치 자체가 노드 OS 안에 있어서, 노드를 장악하면 보안 장치를 끄거나 바꿀 수 있다.

spoofed SIDs
⑧ **SID 위조:** 공격자가 허용된 역할(backend)의 SID를 달고 보내면, 받는 쪽 노드의 Cilium은 정상 패킷으로 믿고 통과시킨다.

원문 4쪽

secure switch
① **secure switch(그림 3):** L3/L4 정책과 신원(SID·IP) 위조 검사를 하는 HNIC의 첫 단계.

Limited policy enforcement performance
② **한계 2:** L7 검사를 위해 커널(eBPF) → **사용자 공간(Envoy)으로 패킷을 복사·왕복하느라 느리고 CPU를 많이 쓴다.**

73% spike in CPU usage
③ **측정:** Cilium으로 L7 정책을 켜면 평균 지연 2배, 서버 노드 CPU 사용 73% 증가.

restricted mode
④ restricted mode(BlueField의 zero-trust 모드): 호스트가 NIC 내부 설정·펌웨어·프로그램에 접근하지 못하게 막는 하드웨어 기능.

HNIC), and a
⑤ **HNIC:** 각 노드의 SmartNIC 위에서 도는 HardWhale 보안 스택. 하드웨어 엔진 3개(secure switch, inspection engine, policy engine)로 구성.

centralized manager
⑥ **manager:** 마스터 노드에서 도는 중앙 관리자. 컨테이너가 생기거나 바뀌면 정책을 계산해 각 HNIC로 보낸다.

physical isolation layer
⑦ **핵심:** 노드 부팅 시 restricted mode를 켜서 호스트 OS와 NIC 사이에 물리적 격리층을 만든다.

TLS
⑧ **TLS:** 통신을 암호화하고 상대를 인증하는 프로토콜(HTTPS의 S). 정책 업데이트 패킷을 보호한다. → **한화비전 카메라-클라우드 연결도 TLS 1.**2/1.3.

원문 5쪽

secure boot [14]
① **secure boot:** 부팅 때 펌웨어·OS·프로그램의 서명을 검증해서 변조된 소프트웨어는 실행하지 않는다. → **EN683 보안 부트와 같은 개념.**

By transitioning to restricted mode following a secure boot
② **보장 사슬:** secure boot로 시작 시점의 무결성을, restricted mode로 실행 중 무결성을 지킨다. 부팅부터 런타임까지 이어진다.

Limited attack surface
③ **공격 표면 축소:** HNIC는 노드의 파일 시스템 등 자원을 쓰지 않고, NIC 제어 인터페이스도 노드에서 막힌다.

HardWhale agent
④ **agent:** 노드 OS에서 도는 보조 소프트웨어. 같은 노드 안 컨테이너 사이 트래픽도 NIC를 거치게 연결한다.

(CNIs)
⑤ **CNI:** 쿠버네티스가 컨테이너를 만들 때 네트워크를 붙여 주는 플러그인 규격. Cilium, Calico가 대표 CNI.

virtual function (VF)
⑥ **SR-IOV VF:** 물리 NIC 하나를 여러 개의 가상 NIC(Virtual Function)로 나눠서 컨테이너에 직접 꽂는 기술. 커널 브리지를 건너뛴다.

spoofing prevention check
⑦ **스푸핑 방지:** VF가 컨테이너가 보낸 패킷의 출발지 주소가 그 컨테이너 주소와 같은지 확인하고 다르면 버린다.

Security implications of agents
⑧ **한계:** agent는 노드 OS에 있으므로 N-comp에서 공격받는다. VF를 떼어 버릴 수 있다. 그래도 다른 노드로 가는 패킷은 HNIC가 걸러서 피해가 그 노드에 갇힌다.

per-node basis
⑨ **노드 단위 배포:** 각 HNIC는 자기 노드 컨테이너에 관련된 정책만 갖는다. NIC 메모리가 작기 때문.

Pre-filtering
⑩ **pre-filtering:** 헤더 파싱, 연결 추적, 관련 없는 패킷 걸러내기를 하는 첫 파이프라인 단계.

원문 6쪽

decapsulation
① **decapsulation:** VXLAN으로 싸서 온 패킷의 바깥 헤더를 벗겨 원래 컨테이너 패킷을 꺼내는 일.

node-level spoofing check
② **노드 단위 위조 검사:** 각 노드는 마스터가 정해 준 주소 대역만 쓴다. 대역 밖 출발지 주소는 버린다. VF 검사가 뚫려도 노드 단위에서 한 번 더 막는다.

stateful
③ **stateful 검사:** 연결(TCP 세션) 상태를 기억해서, 이미 허용된 연결의 후속 패킷은 L3/L4 검사를 건너뛴다.

match-action tables
④ **match-action table:** 패킷 필드가 표의 키와 맞으면(match) 정해진 동작(허용·차단)을 하는 하드웨어 스위칭 표 구조.

SID spoofing
⑤ **SID 위조 차단:** SID 표에 없는 주소이거나, 노드에서 나가는 패킷에 이미 SID가 박혀 있으면 버린다.

per-SID policy parti-
⑥ **SID별 정책 분할:** 정책 표를 SID마다 따로 둬서 조회할 표가 작아지고 빠르다.

Receive-Side Scaling
⑦ **RSS:** 패킷 헤더 해시로 큐를 골라 여러 코어에 나눠 주는 기법. 같은 연결은 항상 같은 코어로 가서 캐시 효율이 좋다.

out-of-order (OoO)
⑧ **OoO 패킷:** 순서가 뒤바뀌어 도착한 TCP 조각. 순서를 맞춰야 패킷 여러 개에 걸친 공격 패턴을 찾을 수 있다.

regex accelerators
⑨ **핵심 가속:** NIC에 내장된 정규식(regex) 하드웨어 가속기로 페이로드를 line rate로 검사한다. 예: GET /user/숫자 경로만 허용.

detection signatures
⑩ **탐지 시그니처:** 알려진 익스플로잇의 바이트 패턴. 페이로드 안에서 찾으면 버린다. IDS(Snort 등)의 방식과 같다.

원문 7쪽

policy engine
① **policy engine:** 매니저가 보낸 정책 업데이트를 실시간으로 반영하는 HNIC의 세 번째 엔진.

eavesdrop-
② **업데이트 통로 위협:** 도청, 위·변조 패킷 주입. TLS와 비밀값 검증으로 막는다.

concurrent pol-
③ **동시 업데이트:** 코어마다 업데이트 큐를 두고 같은 표 항목은 항상 같은 코어가 맡게 해서 락(lock) 없이 병렬로 고친다.

lines of C code
④ **구현 규모:** HNIC는 C 4천 줄(NVIDIA DOCA/Mellanox API), agent·manager는 Python 1,500줄.

ARM cores
⑤ **ARM 코어:** BlueField-2 안의 범용 CPU. 보안 모듈이 여기서 돌고, 가속기를 호출한다. **→ EN683도 ARM 기반 SoC라 같은 툴체인 감각이 쓰인다(추정).**

eBPF programs [18]
⑥ **TC eBPF:** 리눅스 트래픽 제어(tc) 계층에 붙이는 eBPF 프로그램. 노드 내부 위조 검사를 VF마다 이것으로 구현했다.

Q1: Can HardWhale protect
⑦ **보안 평가 질문 1:** 노드가 뚫린 상태에서 다른 노드 컨테이너를 지킬 수 있는가.

Q2: Can HardWhale securely
⑧ **보안 평가 질문 2:** 노드가 뚫린 상태에서 정책을 안전하게 유지·갱신할 수 있는가.

Kubernetes v1.27.3
⑨ **쿠버네티스:** 여러 노드에 컨테이너를 배치·관리하는 오케스트레이터. 실험은 v1.27.3, Ubuntu 22.04, Cilium.

원문 8쪽

with a malicious one
① **Cilium 우회:** 노드를 장악하면 프록시를 검사 안 하는 가짜로 바꿀 수 있다. 소프트웨어 방식의 근본 한계.

SID spoofing attacks
② **그림 7:** IP 위조와 SID 위조 패킷을 모두 차단한다.

exploit packets
③ **결과:** HNIC가 시그니처로 익스플로잇 패킷만 버리고 정상 패킷은 통과시킨다.

buffer
④ **버퍼 오버플로:** 정해진 크기보다 긴 입력으로 메모리를 덮어써서 프로그램 흐름을 빼앗는 공격. 실험은 memcached 취약점 CVE-2016-8704.

Regardless of VFs
⑤ **결과:** agent가 VF를 떼어도 노드 밖으로 나가는 패킷은 HNIC가 걸러서 다른 노드는 안전하다.

Calico [9]
⑥ **Calico:** iptables 기반의 대표 CNI. 역시 정책이 노드 OS에 있다.

iptables rules
⑦ **iptables:** 리눅스 커널의 전통적인 패킷 필터 규칙. 루트 권한이면 지우거나 바꿀 수 있다.

원문 9쪽

invalid secrets
① **결과:** 잘못된 비밀값을 단 정책 업데이트 패킷은 거부한다. 정책 무결성 보장.

encrypted and authenticated through TLS
② **암호화+인증:** 내용을 못 보게 하고(기밀성), 위조하면 드러나게 한다(무결성).

does not forward the used
③ 업데이트 패킷은 노드 OS로 넘기지 않는다. 노드는 정책이 바뀐 사실조차 모른다.

40GbE
④ **실험 환경:** 노드 2대(Xeon 4114, 64GB RAM)를 40Gbps 이더넷으로 연결.

wrk2
⑤ **wrk2:** 일정한 요청률(RPS)로 HTTP 부하를 거는 벤치마크 도구. 지연 분포 측정에 쓴다.

2.3x faster average latency
⑥ **결과:** 저부하(30K RPS)에서 평균 지연 2.3배 빠름.

P99
⑦ **P99 지연:** 요청 100개 중 가장 느린 1개 수준의 지연. 꼬리 지연을 본다. HardWhale 2.489ms, Cilium 12.4ms(4.9배).

Cilium fails
⑧ 고부하(100K RPS)에서 Cilium은 처리하지 못하고, HardWhale은 기준선보다도 최대 1.8배 빠르다(캡슐화 오프로드 덕분).

원문 10쪽

1.08 (5.51) ms
① **표 I:** 클라이언트 수가 늘어도 HardWhale 평균 지연은 약 1.1ms로 일정, Cilium은 1.6→2.0ms로 늘어난다.

more than 3x
② **처리량:** Cilium 대비 3배 이상. Cilium은 기준선 대비 80% 감소, HardWhale은 7% 감소.

multi-pattern matching
③ **다중 패턴 매칭:** 여러 정규식·문자열을 한 번의 스캔으로 동시에 찾는 알고리즘(Aho-Corasick류). 정책 수가 늘어도 느려지지 않는다.

4-fold
④ **결과:** CPU 사용 4분의 1. Cilium은 73%(그중 약 50%가 사용자 공간 프록시).

PCIe overhead
⑤ **약점:** 같은 노드 안 통신은 PCIe로 NIC를 왕복해야 해서 평균 3.5µs 늘어난다.

Linkerd [34]
⑥ **Linkerd, Istio:** 서비스 메시. 컨테이너마다 사이드카 프록시를 붙여 L7 정책과 mTLS를 처리한다.

up to 5x
⑦ L7 프록시(Linkerd, Istio)를 붙인 CNI와 비교하면 같은 노드 통신도 최대 5배 빠르다.

원문 11쪽

P4 switch
① **P4 스위치:** 패킷 처리 파이프라인을 프로그래밍할 수 있는 스위치 칩. 데이터 평면 오프로드 연구에 자주 쓰인다.

Nitro [40]

② **AWS Nitro:** 가상화 기능을 전용 PCIe 카드로 옮긴 AWS의 하드웨어. 강하지만 AWS 전용이라 일반 클라우드에 쓸 수 없다.

off-the-shelf
③ **차별점:** 시중의 SmartNIC만으로 동작하고 상위 소프트웨어 수정이 필요 없다.

8K policies
④ **한계:** NIC 메모리 때문에 약 100만 흐름, 노드당 정책 8천 개까지만 line rate로 처리한다.

scalability
⑤ **한계:** 노드 안 모든 컨테이너를 SR-IOV VF로 연결해야 해서 노드당 컨테이너 수가 VF 수로 제한된다.

5.7-fold
⑥ **결론:** OS가 뚫려도 공격을 막고 지연을 최대 5.7배 개선.

원문 12쪽

HELIOS
① **참고 [11]:** HELIOS(SoCC 2023)는 같은 저자들의 선행 연구. 하드웨어 보조 클라우드 네트워크 보안 확장.

Zero trust mode
② **참고 [13]:** BlueField DPU zero-trust 모드 = 본문의 restricted mode.